

# BulkBox Lab

*Independent Playwright practice*

Now let's put what we learned in class to the test. In this lab, you'll be testing **BulkBox** (<http://localhost:4200>), a small ordering site for shipping and packing supplies like cardboard boxes, tape, labels, mailers, and related warehouse materials. The site is simple enough to practice on, but it still gives us real testing problems to solve: finding reliable elements, clicking through user flows, checking URLs, reading page content, filtering product results, and eventually reusing authentication.

Before you start writing tests, get the app and the lab project running locally. The setup steps below make sure you are on your own branch, the dependencies are installed, and BulkBox is running before Playwright tries to connect to it.

**Demo login:** buyer / supplies123

## SETUP

Your lab project lives in `workspace/lab2/`. It is already set up with `playwright.config.ts` and `package.json` pointed at BulkBox. A skeleton `ProductsPage` also exists in `tests/pages/products.page.ts`. It is just a bare class with a constructor. You will build it out yourself in Task 4 before using it in the product tests. You do not run `npm init playwright@latest`. Tests run in Google Chrome, which is already installed, so there is no browser download.

### 1. Branch off main:

```
git checkout main
git pull
git checkout -b <lastname>-playwright-lab
```

### 2. Install dependencies, from inside `workspace/lab2/`:

```
npm install
```

### 3. Start BulkBox and leave it running, from `workspace/lab2/`:

```
npm run start:app
```

Confirm `http://localhost:4200` loads. Optional: you can skip this step because Playwright auto-starts BulkBox when you run the tests and reuses it if it is already up. Starting it yourself just makes the first test faster and lets you use `--ui`.

⚠ If you start BulkBox manually, leave it running while you work. Playwright will reuse it if it is already up.

### 4. Run tests:

```
npx playwright test      # all tasks
npx playwright test --ui  # watch / debug
```

Write specs in `tests/`. The starter `day2.spec.ts` and `day3.spec.ts` files have a skeleton per task marked `test.fixme`. Remove `.fixme` and fill in the body as you work each one.

**Note:** Test the BulkBox front end as it is provided. Do not change the app's HTML, labels, attributes, or markup to make a test easier to write or pass. Your work should be in the Playwright tests and page objects only.

## HOW TO TURN IN THE LAB

After setup, you will be working on your own branch for the rest of the lab. When you are finished, push that branch, named `<lastname>-playwright-lab`, to the remote and open a pull request into `main`.

# PHASE 1: CORE TESTS AND PAGE OBJECTS

*Clean markup.*

Let's put what we learned about Playwright tests to work. You'll start with straightforward tests that load a page, locate important content, click a call-to-action, and assert that the app responded correctly. Then you'll build out the `ProductsPage` so the product-page tests can use a shared page object instead of repeating raw locators.

## 1. Load the home page and confirm the hero banner and logo are visible

**Page:** Home (/)

When a user lands on the home page, the hero banner and the BulkBox logo/home link should be visible immediately. Write a test that confirms both elements are visible.

## 2. Use the home page call-to-action to navigate to the products page

**Page:** Home (/) → Products (/products)

From the home page, click the "Shop all products" button. You should land on the products page, and the results heading should be visible. Write a test that performs the click and confirms both outcomes.

## 3. Search from the header and verify the products page reflects the query

**Page:** Home (/) → Products (/products?q=Tape)

Search for Tape using the header search bar. The URL should reflect the search term, and the results count should indicate that matching products were returned. Write a test that performs the search and verifies both outcomes.

## 4. Build the `ProductsPage` page object

**Page:** Products (/products)

A skeleton already exists at `tests/pages/products.page.ts`. It is a class with the method signatures already stubbed out and the bodies left empty. Your job is to fill in each method. Do not write a test for this task yet. Just get the class working so the next tests can use it.

By the end of this task, the page object needs to be able to:

- Select a category.
- Toggle "in stock only."
- Choose a sort order.
- Read back the displayed results count.
- Read the product rows, including the row count and a specific row's price.

## 5. Search, then narrow the results, using the same ProductsPage

**Page:** Products (/products)

Using ProductsPage, search for Tape and confirm the results count is greater than zero. Then, without creating a new page object or re-navigating, enable “in stock only” on that same instance and confirm the count drops by one. Packing Tape model S-425 is out of stock, so it should disappear from the results.

## 6. Sort the filtered results and confirm the order actually changed

**Page:** Products (/products)

Continuing on the same ProductsPage instance from Task 5, with the Tape category and in-stock-only filter still applied, sort by price from low to high. Confirm that the order actually changed by comparing what the first item was before sorting with what it is after sorting. Before sorting, the first item should not be the lowest-priced Tape item. After sorting, the first item should be Masking Tape at \$1.75, the cheapest in-stock Tape item.

## ADVANCED

*The next few are harder: markup you do not fully control, plus authentication.*

### 7. Extend ProductsPage, then add the correct duplicate product to the cart

**Page:** Products (/products)

Your ProductsPage does not yet have a way to locate a row by name and price or click its “Add to cart” button. Add that capability to the class first, then write the test using the instructions below.

BulkBox has two products named “Corrugated Shipping Box.” They share the same product name but have different models and prices. Go to the unfiltered products page and locate both entries. Confirm that two matching products are present, then add the \$1.59 item, model S-4614, to the cart rather than the \$1.29 item. The grid does not provide test ids for this scenario, so use role and text-based locators to narrow the selection before clicking. Confirm that the cart count increases to 1 after the item is added.

### 8. Navigate the category menu, using ProductsPage

**Page:** Any page with the header → Products (/products?category=...)

The category mega-menu in the header has no test ids and uses hashed class names. Add a method to ProductsPage that opens the mega-menu and clicks a specific subcategory link located by role and visible text. Then confirm that the resulting products page reflects that category.

### 9. Optional: Save an authenticated session and reuse it in a separate test

**Page:** Login (/login), then Home (/)

Logging in during every test creates unnecessary repetition. Instead, perform the login once: write a test that logs in as buyer / supplies123, then saves the authenticated session to a file using `page.context().storageState({ path: 'auth.json' })`. Confirm that the file is created successfully.

Then write a second, separate test that uses the saved file instead of logging in again. Set `test.use({ storageState: 'auth.json' })` at the top of the file, go directly to the home page, and confirm the header greets the logged-in user. The test should not interact with the login form.

### 10. Optional: Validate login errors for missing or incorrect credentials

**Page:** Login (/login)

If time allows, submit the login form with both fields empty and confirm that validation errors appear for both the username and password. Then try logging in with the correct username and an incorrect password, and confirm that a general “invalid username or password” error appears.

## COOKBOOK PRACTICE

*Optional examples for additional practice after the required lab is complete.*

This workshop cannot cover every scenario you may need to test with Playwright. To give you additional practice and a reference you can return to after class, we have prepared a cookbook/ folder with worked examples for common UI patterns. These examples are not required for the lab submission, but they are available if you want to explore more scenarios or review them later.

1. Tabs
2. Accordions
3. Toasts
4. Cart remove-confirmation modal
5. Sortable tables
6. Paginated tables
7. Cascading selects
8. Date picker
9. Debounced autocomplete
10. CSV upload
11. Opening a new tab
12. Iframe widget